Project Report

1. Project Overview

For this project, we added a DevOps workflow to Docmost, an existing open-source collaborative documentation platform. We reuse the existing application, tests, and Dockerfile. Our work mainly consists of the GitHub Actions workflow, the Terraform configuration in infra/, docker-compose.yml, and scripts/deploy.sh.

The application is deployed on AWS. Route 53 provides DNS for docmost.click and directs requests to an Application Load Balancer (ALB). The ALB handles HTTPS and forwards the traffic over HTTP to an EC2 instance. Docmost, PostgreSQL, and Redis run on this instance using Docker Compose. We use Docker Compose because the deployment is small and does not need a more complex container orchestration system. PostgreSQL and Redis are only accessible inside the Docker network.

We store persistent data on a separate encrypted EBS volume. Application files, PostgreSQL data, and Redis data are stored in separate directories on this volume. This means that the data remains when containers are replaced. The ALB uses Docmost's existing /api/health endpoint for health checks. This endpoint also checks the connections to PostgreSQL and Redis.

2. CI/CD Workflow

GitHub Actions runs when a pull request is opened or updated and when code is pushed to main. Two CI jobs run in parallel.

The test-and-build job installs the dependencies, builds the shared packages, runs the frontend Vitest tests and a selected set of backend Jest tests, and then builds the complete project. The docker-build job builds the production Docker image using Docmost's existing multi-stage Dockerfile.

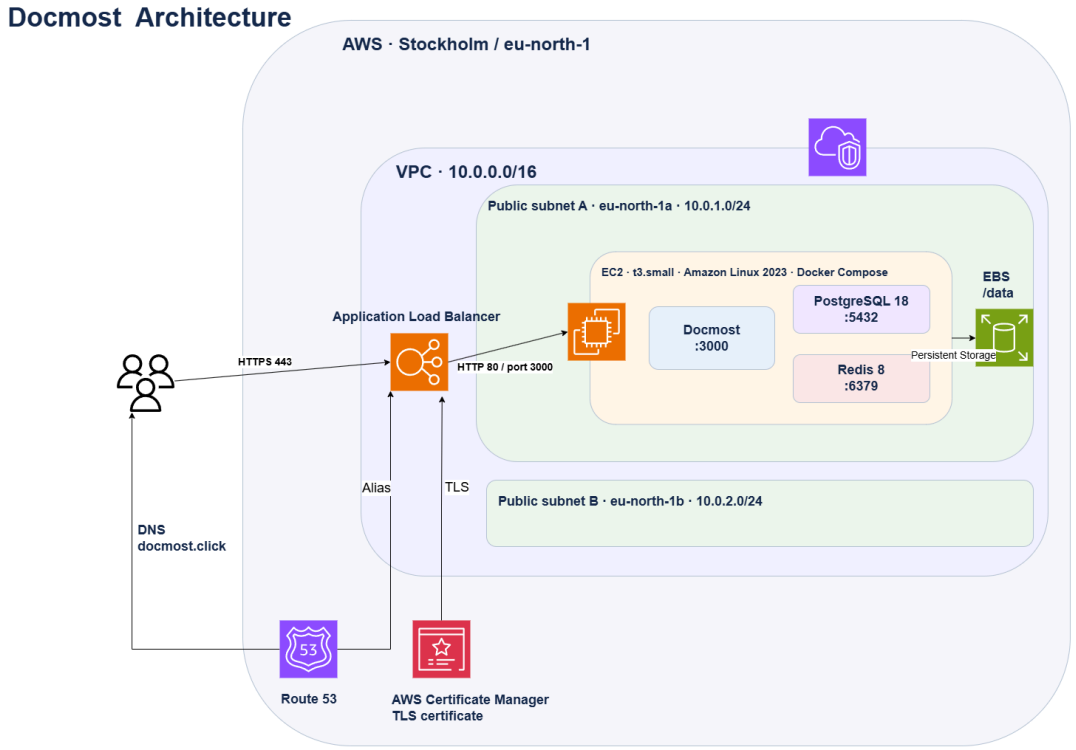
After the image is built, Trivy scans it for vulnerabilities. The job fails if it finds a CRITICAL vulnerability that has an available fix. Other vulnerabilities are reported but do not stop the pipeline.

When code is pushed to main, the image is also published to GitHub Container Registry (GHCR). We use the full Git commit SHA as the image tag, so we can see exactly which version of the source code an image belongs to.

The deployment job only starts after both CI jobs have succeeded. The image is published inside the docker-build job, so it is possible for an image to be pushed to GHCR before test-and-build has finished. If the tests later fail, the image remains in GHCR but is not deployed.

We do not run the complete upstream backend test suite because several legacy Docmost tests already fail with the current test setup. Instead, we use a subset of backend tests that passed consistently during our baseline checks. This keeps existing upstream test failures from blocking our pipeline while still giving us regression tests for the covered parts of the application.

3. Infrastructure and Deployment



We use Terraform to define the AWS infrastructure, including the network, EC2 instance, load balancer, EBS volume, HTTPS resources, security groups, and IAM configuration. The Terraform state is stored in the docmost-prod workspace in Terraform Cloud to enable both team members to apply the change.

Infrastructure changes are separate from application deployment. Our GitHub Actions workflow therefore does not automatically run terraform plan or terraform apply.

For deployment, GitHub Actions connects to AWS using OIDC and assumes a temporary IAM role. We do not store long-lived AWS access keys in GitHub. The deployment role can send Systems Manager (SSM) commands to the Docmost EC2 instance and check their status. The EC2 instance has its own IAM role for SSM and for reading application parameters.

SSM runs scripts/deploy.sh from the commit that is being deployed. The script gets the docker-compose.yml file from the same commit and uses the GHCR image with the same commit SHA. In this way, the source code, Compose configuration, and Docker image all refer to the same version.

The application secret and database password are read from AWS Systems Manager Parameter Store. The script writes them to a runtime environment file with permission 600. These parameters have to be created separately and are not managed by our Terraform configuration.

The deployment script first validates the Compose configuration and pulls the required image. It then runs docker compose up -d. GitHub Actions checks the SSM command until it succeeds, fails, or reaches the polling timeout.

4. Use of AI Tools

We mainly used ChatGPT as a learning and debugging tool during the project. It helped us understand the architecture of the existing Docmost project and learn basic AWS concepts that were new to us, such as OIDC and IAM. We also asked ChatGPT to write small tutorials on how to implement Terraform. A typical prompt could be “Give me a tutorial on creating ALB service, explain in plain language, don’t write the code directly but give me small demos and also give alternatives and list tradeoffs.”

We also used ChatGPT to help inspect error messages when terraform apply failed and to learn how to add HTTPS to our deployment.

5. Limitations

We run Docmost, PostgreSQL, and Redis on the same EC2 instance. This makes the deployment simple enough to manage with Docker Compose, but it also means that the EC2 instance is a single point of failure. Running the application and stateful services separately could improve availability, but would also make the infrastructure more complex.

The EBS volume keeps persistent data outside the containers. We also use Terraform prevent_destroy to reduce the risk of accidentally deleting the volume through Terraform. However, this is not a backup. We currently do not have scheduled backups or restore testing.

There is also a limitation in how we verify deployments. A deployment is considered successful when the SSM command and docker compose up -d complete successfully. The deployment script does not wait for /api/health to become healthy afterwards. Because of this, the pipeline could report a successful deployment even if the application fails during startup. A future improvement would be to add a health check with a timeout after starting the containers.

Finally, production deployments are not serialised. If two workflow runs reach the deployment job at the same time, they could overlap. We also do not currently have centralised production logging or alerting.